

# Estructuras de Datos — Quiz 03

Dynamic Arrays and Amortized Analysis · analysis.pdf · Semestre 2026-2

**Nombres:** Kevin Javier Gonzalez, Iván Felipe Maluche Suarez, Angelica Pascagaza Vega **Documento:** 1093432420 **Grupo:** 3

## Notación y modelo de costo

### P1 — Arreglo dinámico desde cero

a) **Implementación.** En el archivo de `DynamicArray.java` se encuentra la implementacion del arreglo dinamico.

b) **Invariantes.**

I1.

$$0 \leq size \leq capacity$$

I2.

$$capacity \geq 1$$

I3.

$$index \in [0, size), data[size] == null$$

c) **Complejidad worst-case.**

| Operación               | Worst-case  | Justificación                                                                                                                                                                                    |
|-------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>get</code>        | $\Theta(1)$ | El metodo solamente comprueba si el index es valido y devuelve el dato, debido a que el array por defecto ya lo hace sin necesidad de recorrer nada cuesta $O(1)$                                |
| <code>set</code>        | $\Theta(1)$ | Similar a <code>get</code> , es una busqueda de costo 1 mas en este caso se asigna al valor un dato manteniendo constante la cota                                                                |
| <code>append</code>     | $O(n)$      | Debido al <code>resize</code> en el peor caso es necesario construir un nuevo array y dato por dato del anterior asignarlo en el mismo.                                                          |
| <code>removeLast</code> | $\Theta(1)$ | Tal como el caso de <code>get</code> y <code>set</code> consiste en guardar el ultimo valor, asignarle un nulo y disminuir <code>size</code> , siendo operaciones que me dan una cota constante. |

d) **Worst-case vs. amortizado.**

Al realizar el `append` siempre que  $size < capacity$  el coste de operacion sera "barata", lo cual permite que cuando pase el caso en que  $(size = capacity)$  el costo de `resize` se amortizara en el total.

### P2 — Snake sobre DynamicArray con desplazamiento

a) **Costo de `removeTail`.**

Al llamar `removeTail` en esa implementacion es necesario eliminar la cola y luego reasignar los valores que quedan una posicion hacia atras, esto implica un costo  $O(n)$  (con  $n$  siendo en este caso el `size`) al tener que recorrer el array .

b) **Costo total de  $m$  movimientos.**

Al ser de longitud  $k$  habria un costo de  $O(k)$  lo cual al realizarse  $m$  veces nos daria un costo total de  $O(km)$ .

c) **Validez de la afirmación.** Esto es falso, `move()` no es amortizado porque en cada operacion de `move()` tengo que hacer  $n$  (`size`) actualizaciones de posicion de la serpiente lo que hace que en promedio

sea  $\frac{O(n \cdot n)}{n} \rightarrow O(n)$  veces el costo amortizado, a diferencia del caso de append donde gracias a la distribución entre operaciones caras y baratas el coste se amortigua.

#### d) Bottleneck.

El bottleneck en esta implementación consistiría en que debido a que en cada movimiento que la serpiente no consuma una manzana (lo cual pasará frecuentemente) será necesario ejecutar un RemoveTail, el cual como se explicó en el punto anterior es  $O(n)$ . Esto culminará en que esta operación básica y siempre necesaria para todo el loop jugable tenga un costo considerable y pueda generar problemas de estabilidad al ejecutarse en casi todo momento.

### P3 — Rediseño: representación interna

#### a) Variables de estado.

- data:
- start:
- size:
- capacity:

#### b) Mapeo lógico $\rightarrow$ físico.

$$\text{phys}(i) =$$

#### c) Operaciones.

- addHead:
- removeTail:
- get:

#### d) resize.

#### e) Invariantes y corrección.

S1.  
S2.  
S3.  
S4.

#### f) Traza.

| Mov.         | Arreglo físico | start | size | capacity | ¿resize? | Orden lógico |
|--------------|----------------|-------|------|----------|----------|--------------|
| inicial      |                |       |      |          |          |              |
| N1 (no come) |                |       |      |          |          |              |
| N2 (come)    |                |       |      |          |          |              |
| N3 (no come) |                |       |      |          |          |              |
| N4 (no come) |                |       |      |          |          |              |
| N5 (come)    |                |       |      |          |          |              |
| N6 (no come) |                |       |      |          |          |              |

### P4 — Análisis amortizado

#### a) Aggregate Method.

#### b) Suma geométrica.

#### c) Costo total y amortizado de addHead.

d) Papel de  $m$  y  $g$ .

e) Método del potencial.

$$\Phi(\text{size}, C) =$$

- addHead sin resize:
- resize:
- removeTail:

## P5 — The Adversarial Snake

a) El adversario no cambia el número de copias.

b–e) Comparación de políticas.

---

| Política             | # Resizes | Total copias | Costo amortizado<br>por crecimiento | Espacio sin usar tras<br>resize |
|----------------------|-----------|--------------|-------------------------------------|---------------------------------|
| <hr/>                |           |              |                                     |                                 |
| $C + 1$              |           |              |                                     |                                 |
| $2C$                 |           |              |                                     |                                 |
| $\lceil 1.5C \rceil$ |           |              |                                     |                                 |

---

f) Trade-off.

Familia  $C_{\text{new}} = \lceil \alpha C \rceil$ .

- Valores de  $\alpha$ :
- Progreso ( $C_{\text{new}} > C$ ):
- Cota superior de copias:
- Cota inferior de copias:
- Costo amortizado:
- Necesidad de  $\alpha > 1$  constante: